

Technical requirements

The chapter examples can be run on the *chapter_10* Docker image that you can find in the book's GitHub repository: <https://github.com/PacktPublishing/Learn-PostgreSQL-Second-Edition>.

Understanding roles

In *Chapter 3, Managing Users and Connections*, you saw how to create new roles, a stereotype that can act either as a single user or a group of users. The `CREATE ROLE` statement was used to create the role, and you learned about the main properties a role can be associated with.

This section extends the concepts you read about in *Chapter 3, Managing Users and Connections*, introducing the more interesting and security-related properties of a role.

Just as a quick reminder, the synopsis for creating a new role is the following:

```
CREATE ROLE name [ [ WITH ] option [ ... ] ]
```

The name assigned to the role has to be unique within the whole cluster.

An option can be indicated in a positive form, that is, associating a property with a role, or in a negative form with the `NO` prefix, which removes a property from a role. Some properties are not assigned to new roles by default, so you should take your time and consult the documentation of the `CREATE ROLE` statement in order to see what the default value is for each property. If you are in doubt, associate explicitly the properties you need and negate those you absolutely don't want your roles to have.

Properties related to new objects

There are two main capabilities that a role can acquire in order to create new objects, and both should be given only to trusted parties:

- `CREATEROLE` allows a role to create and manipulate other roles (and therefore database accounts and groups).
- `CREATEDB` allows a role to create other databases within the cluster.

By default, if not specified explicitly, a new role is created without such capabilities, hence:

```
postgres=# CREATE ROLE luca;  
Is wholly equivalent to the following command:  
postgres=# CREATE ROLE luca  
          WITH NOCREATEROLE  
          NOCREATEDB;
```



In this chapter you will see interleaved commands entered by regular users, with a `forumbd=>` command prompt, and commands entered by the database administrator, with a `forumbd=#` prompt. If not explicitly specified, all the examples related to granting and revoking permissions will be run as the `forum` database user.

Properties related to superusers

With the `SUPERUSER` property, a role is created as a cluster administrator, that is, a role that has every right on every object within the cluster, most notably the capability to add, remove, and change users; change the PostgreSQL configuration; terminate user connections; and halt the cluster.

It is possible to have as many superusers as you want in a cluster. However, being a class of users without any restrictions, it is a good habit to avoid giving all the permissions to untrusted users unless it is strictly necessary. Preventing the usage of superuser roles whenever possible is another good habit that can help prevent accidental damage to a cluster and its data.

Properties related to replication

The `REPLICATION` property is used to specify that the new role will be able to use the replication protocol, a particular networking protocol that PostgreSQL uses to replicate data from one cluster to another.

`REPLICATION` is an option that allows a role to access all the data within the cluster without any particular restriction. Therefore, it is usually granted to just those roles used for replication.

Due to its security implications, if not specified otherwise, the `NOREPLICATION` option is set.

Properties related to RLS

`RLS` is a policy enforcement mechanism that prevents certain roles from gaining access to specific tuples within specific tables. In other words, it applies security constraints at the level of table rows, hence the name **row-level security**.

There is a single option that drives RLS: `BYPASSRLS`. If the role has such an option, the role **bypasses** (which means it is not subjected to) all security constraints for every row within the cluster. The default for this option, as you can imagine, is to negate it (that is, `NOBYPASSRLS`) so that roles are subjected to security enforcement whenever possible.

It is important to note that cluster superusers are always able to bypass RLS policies.

You will learn more about RLS in the *RLS* section of this chapter.

Changing properties of existing roles: the ALTER ROLE statement

As you can imagine, once they have been created, roles are not immutable: you can add or remove properties to or from a role by means of the ALTER ROLE statement. The synopsis for the statement is very similar to the one used to create a role, and is as follows:

```
ALTER ROLE name [ [ WITH ] option [ ... ] ]
```

Here, name is the unique role name and the options are specified in the exact same manner as in the CREATE ROLE statement.

As an example, imagine you want to provide the luca role with the capabilities to create databases and new roles. You can issue two ALTER ROLE statements or combine the options as follows:

```
forumb=# ALTER ROLE luca WITH CREATEDB;
ALTER ROLE

forumb=# ALTER ROLE luca WITH CREATEROLE;
ALTER ROLE

-- same as the above two statements
forumb=# ALTER ROLE luca CREATEROLE CREATEDB;
ALTER ROLE
```

And if you, later on, change your mind, you can remove one or both options by assigning the negated form:

```
forumb=# ALTER ROLE luca NOCREATEROLE NOCREATEDB;
ALTER ROLE
```

The ALTER ROLE statement is always executable by a cluster superuser, but can also be executed by a non-superuser role that has the CREATEROLE option (that is, can create, and therefore manipulate, other roles), as long as the statement is applied to a non-superuser role.

Renaming an existing role

The ALTER ROLE statement also allows for a change in the name of the role: the RENAME clause allows for a role to be substituted by another unique role name. As an example, let's change a role's *short* username to a longer one:

```
forumbd=# ALTER ROLE luca RENAME TO fluca1978;
ALTER ROLE
```

Obviously, you cannot rename an existing role using a destination role name that is already in use.

It is possible to rename the role back to its previous value with the same command:

```
forumbd=# ALTER ROLE fluca1978 RENAME TO luca;
ALTER ROLE
```

SESSION_USER versus CURRENT_USER

The `ALTER ROLE` statement operates on an existing role, specified by its role name. It is, however, possible to refer to the current role with two particular keywords: `SESSION_USER` and `CURRENT_USER`.



Mind the usage of `user` in the `SESSION_USER` and `CURRENT_USER` special keywords. They still refer to the concept of *role*, but for backward compatibility, they use the *user* nomenclature. While there is a `CURRENT_ROLE` keyword, there is not an equivalent `SESSION_ROLE` one.

`SESSION_USER` is the role name of the role that is connected to the database, which means the user that has opened a session to the database.

`CURRENT_USER` (or `CURRENT_ROLE`) is the role name of the role that has been explicitly set by a `SET ROLE` statement.

Once a connection is established, the two keywords refer to the very same role that opened the connection (that is, the one specified in the connection parameters or the connection string). If the role performs an explicit `SET ROLE` operation, `SESSION_USER` remains unchanged, while `CURRENT_USER` reflects the last specified role.

Let's see this in action. Suppose the user `luca` opens a connection to the database. In the beginning, both `SESSION_USER` and `CURRENT_USER` hold the same value:

```
$ psql -U luca forumbd

forumbd=> SELECT current_user, session_user;
 current_user | session_user 
-----+-----
      luca    |      luca    
(1 row)
```

Assume the `luca` role is a member of a group named `forum_stats`, so it is possible to perform an explicit *transformation* to such a role:

```
forumdb=> SET ROLE forum_stats;
SET
forumdb=> SELECT current_user, session_user;
current_user | session_user
-----+-----
forum_stats  | luca
(1 row)
```

As you can see, after the `SET ROLE` statement, `CURRENT_USER` changed its value to reflect the role the user is actually playing, while `SESSION_USER` holds the *original* value that the user used to connect to the database.

To summarize, `CURRENT_USER(CURRENT_ROLE)` tracks the role that is currently running, while `SESSION_USER` holds the role that the database connection was opened with.

Per-role configuration parameters

Along with role properties and granted permissions, roles can also be attached with some configuration parameters that can document their usage. Essentially, it is possible to attach a list of `SET` commands to a role so that every time the role connects to a database, such commands are implicitly executed.

Let's say the user `luca` executes a `SET` command for the `client_min_messages` value every time they connect to the database:

```
$ psql -U luca forumdb

forumdb=> SET client_min_messages TO 'DEBUG';
SET
```

This can be annoying and, most importantly, risky. The user could forget to execute the `SET` command, which they need for the connection to work as expected. It is possible to change the role so that they execute the `SET` command automatically as soon as a connection is established:

```
forumdb=# ALTER ROLE luca
        IN DATABASE forumdb
        SET client_min_messages TO 'DEBUG';
```

ALTER ROLE

And now, every time the luca role connects to the forumdb database, the SET command is automatically executed:

```
$ psql -U luca forumdb

forumdb=> SHOW client_min_messages;

 client_min_messages
-----
 debug
(1 row)
```

The general syntax for changing runtime parameters for a role is as follows:

```
ALTER ROLE name IN DATABASE dbname SET parameter_name TO parameter_value
```

Here, you have to specify the role name or the special keyword ALL for every existing role, the database name, and the name and value of the parameter you want to change.

It is also possible to discard any per-role configuration with the RESET ALL clause, as in the following example:

```
forumdb=# ALTER ROLE luca
          IN DATABASE forumdb
          RESET ALL;

ALTER ROLE
```

Inspecting roles

There are different ways to inspect existing roles and get information about their properties. One quick approach, as already seen in Chapter 3, *Managing Users and Connections*, is to use the \du command in psql:

```
forumdb=> \du

                                List of roles

 Role name | Attributes 
-----+-----
--
 book_authors | Cannot login
  enrico      |
```

forum	
forum_admins	Cannot login
forum_emails	No inheritance, Cannot login
forum_stats	No inheritance, Cannot login
luca	1 connection
postgres	Superuser, Create role, Create DB, Replication, Bypass RLS

The Attributes column provides a mnemonic description of the role properties: as an example, the luca role is limited to a single connection.

Besides the special commands of `psql`, the superuser can always query the system catalog to get information about the existing roles. The main entry point is the table `pg_authid`, which contains one row per existing role with a column that reflects every property of the role (that is, what you defined via the `CREATE ROLE` or `ALTER ROLE` statements), for example:

```

forumdb=# \x
Expanded display is on.
forumdb=# SELECT * FROM pg_authid WHERE rolname = 'luca';
-[ RECORD 1 ]--+-+-----
oid           | 16384
rolname       | luca
rolsuper      | f
rolinherit    | t
rolcreaterole | f
rolcreatedb   | f
rolcanlogin   | t
rolreplication | f
rolbypassrls  | f
rolconlimit   | 1
rolpassword   | SCRAM-SHA-256$4096:...f2QU/7KAVM=
rolvaliduntil |

```

Every role has a unique name and an OID value, which represents the role as a numerical value. This is similar to how users are represented in the Unix system (and many others) where the numerical value of a role is only used internally.

Many of the role properties have a Boolean value, where `f` means false (that is, no option) and `t` means true (that is, with an option).

For instance, in the preceding example, you can see that `rolcreatedb` is false, which means that the role has been created (or altered) with the `NOCREATEDB` option.

The role password (`rolpassword` field) is expressed as a hash, with the identifier of the algorithm used (in the above, `SCRAM-SHA-256`).

There is another possible catalog, named `pg_roles`, that displays the same information about `pg_authid`:

```

forumdb=> SELECT * FROM pg_roles WHERE rolname = 'luca';
-[ RECORD 1 ]--+-+-----
rolname       | luca
rolsuper      | f
rolinherit    | t
rolcreatorole | f
rolcreatedb   | f
rolcanlogin   | t
rolreplication | f
rolconlimit   | 1
rolpassword   | *****
rolvaliduntil | 
rolbypassrls  | f
rolconfig     | 
oid           | 16384

```

Why do we need two similar views of the same data? Only cluster superusers can query `pg_authid`, while every user can query `pg_roles`, since there is no risk of the role password being revealed (as you can see the password field has been masked out).

What about group membership? You can query the special `pg_auth_members` catalog to get information about what roles are members of what other roles. As an example, the following query provides a list of groups:

```

forumdb=> SELECT r.rolname, g.rolname AS group,
               m.admin_option AS is_admin
FROM pg_auth_members m
JOIN pg_roles r ON r.oid = m.member
JOIN pg_roles g ON g.oid = m.roleid
ORDER BY r.rolname;

```

rolname	group	is_admin
enrico	book_authors	f
enrico	forum_admins	f
luca	forum_stats	f
luca	book_authors	f
pg_monitor	pg_read_all_settings	f
pg_monitor	pg_read_all_stats	f
pg_monitor	pg_stat_scan_tables	f
test	forum_stats	f

(8 rows)

Roles that inherit from other roles

We already saw in *Chapter 3, Managing Users and Connections*, that a role can contain other roles, therefore behaving as a group.

When a role becomes a member of another role, it gets all the permissions of the containing (group) role. However, there are cases where such privileges are dynamically granted, that is, the member role will have the privileges transparently, and cases where the privileges will be granted statically, that is, the member role needs to explicitly become the group role in order to use its privileges. The `INHERIT` property of a role discriminates how roles can use privileges by default. The `GRANT` statement has an optional `WITH INHERIT` clause that can be specified when a role is added to a group: if the clause has a true value, then the added role will immediately and dynamically get all the permissions of the group, otherwise it will not.

If the `WITH INHERIT` option is not specified, then the role's `INHERIT` property will be implicitly used. In order to understand the difference and the implication, let's see how the `forum_admins` and `forum_stats` roles could have been built:

```

forumdb=# CREATE ROLE forum_admins WITH NOLOGIN;
CREATE ROLE

forumdb=# CREATE ROLE forum_stats WITH NOLOGIN;
CREATE ROLE

forumdb=# REVOKE ALL ON forum.users FROM forum_stats;
REVOKE

```

```
forumdb=# REVOKE ALL ON forum.users FROM forum_admins;
REVOKE

forumdb=# GRANT ALL ON SCHEMA forum TO forum_admins;
GRANT

forumdb=# GRANT USAGE ON SCHEMA forum TO forum_stats;
GRANT

forumdb=# GRANT ALL ON forum.users TO forum_admins;
GRANT

forumdb=# GRANT SELECT (username, gecos) ON forum.users TO forum_stats;
GRANT

forumdb=# GRANT forum_admins TO enrico;
GRANT ROLE

forumdb=# GRANT forum_stats TO luca;
GRANT ROLE
```

First of all, the two roles are created without the capability to log in directly; this is because we don't want the group to be used as a user itself. Rather, we want the users belonging to the group to be able to log in to the database. Then, we remove, by means of `REVOKE`, all the permissions from `forum_stats` for the table `users`.

This is a good habit: revoking all the permissions allows you to clearly set only the permissions you really want, without accidentally assigning permissions you don't want to your group. Similarly, we provide all the permissions to the `forum_admins` role for the `users` table. Then, we *tune* the permissions giving all the permission on the `users` table to the `forum_admins`, and only a `SELECT` permission over two columns to the `forum_stats` group. Last, we make `enrico` a member of the `forum_admins` role by `GRANT`-ing the latter to the former, and similarly, we do so with `luca`, who becomes a member of the `forum_stats` group.

It is quite simple to see how the `enrico` role can perform what the `forum_admins` role allows him to do on the `users` table: being a member of the `forum_admins` group, the `enrico` role can perform any action against the `users` table.